Error Handling

Module 1: Python Foundations

```
try:
    int("abc")
except ValueError:
    print("Not a number")
```

By the End of Today, You'll Be Able To…

1

Write a `try/except` that catches a real error

2

Recognize common error types on sight

3

Decide: check first, or try first?

You'll Know These by the End of Today

Exception

an error event

`try / except`

attempt + handle

`ValueError`

bad value

`TypeError`

wrong type

…And These Three, Too

`NameError`

unknown name

`IndexError`

missing position

`KeyError`

missing key

Reading Recap: Try It, Catch It, Keep Going

Today's Reading: Error Handling

A hotel program books room requests one line at a time. One guest types "4 nights" and every booking after that line stops. Reopen the reading: Try It, Catch It, Keep Going: Error Handling

The Scenario: A Hotel Booking Program

5 room requests

`Okafor|2|double` is guest, nights, room. `int()` turns the nights into a number.

No `try/except`

`int("4 nights")` crashes on Batista's line. Only 2 guests get booked. Lindqvist and Moreau are never reached.

With `try/except`

Batista is skipped with a message. Lindqvist and Moreau still get rooms: 4 of 5 booked.

READING RECAP

Key Terms and Code From the Reading

Exception

`int("4 nights")` raises one. Uncaught, the program stops at that line.

`try / except`

`try` holds the risky step. `except ValueError:` runs only if that error is raised.

Named errors

`ValueError`: bad value. `KeyError`: missing key. `IndexError`: no such position.

Bare `except:`

`except:` with no error name catches every error, even a typo. It hides real bugs.

READING CHECK

From the reading: `int("4 nights")` runs inside `try`, with `except ValueError` below it. What happens?

A A traceback. The program stops.

B `except` runs. The loop keeps going.

C It prints `Nights: 4 nights`

ANSWER KEY

Answer Key: Caught, Then Kept Going

Correct answer: B. `except` runs. The loop keeps going.

`int("4 nights")` raises a ValueError. Python leaves `try` at that line, runs `except ValueError`, then moves on to the next value.

READING CHECK

The `try` converts nights, then reads `rates[room].rates` has no `"penthouse"` key. Which block runs?

A `except ValueError`

B Both blocks, one after the other

C `except KeyError`

ANSWER KEY

Answer Key: Name the Error You Expect

Correct answer: C. `except KeyError`

`rates["penthouse"]` has no entry, so it raises a KeyError. Only the `except` that names it runs. Nights was a real number, so `except ValueError` never fires.

Your Questions From the Reading

Pair up

1 min: tell a partner one question you still have.

Ask

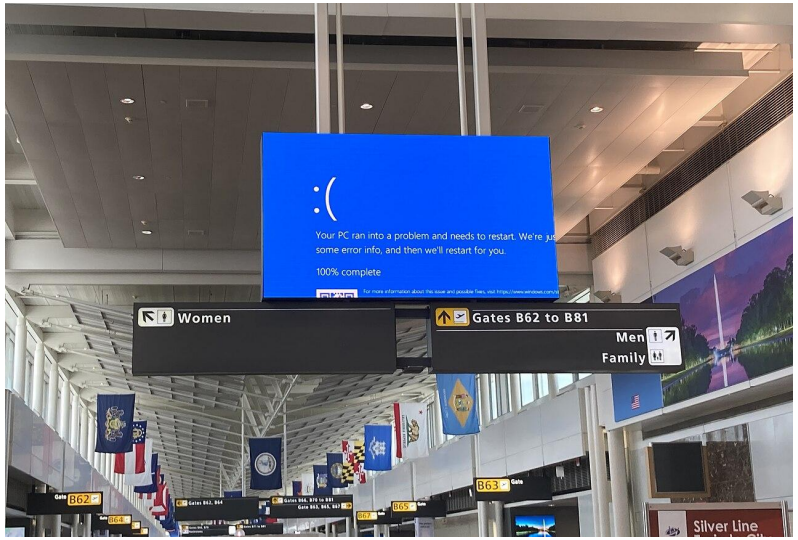
We take 2 or 3 questions now.

Park it

Bigger ones wait for today's matching slide.

IN THE NEWS

One Missing Check Took Down 8.5 Million Computers



Dulles Airport, July 19, 2024. Photo: reivax, CC BY-SA 2.0, via Wikimedia Commons

Code expected 21 fields, got 20
Code read field 21 without checking it
Airports, hospitals and banks went dark

Your code is handed data from somewhere else. What do you check before you trust its shape?

Source: CrowdStrike, Channel File 291 Root Cause Analysis, August 6, 2024

What If You Just Try It?

The last lesson's `isdigit()` check happens before the risky step. Today's tool handles what happens if you skip straight to it.

Not Every Bad Value Announces Itself First

Before

Check the shape first.

Not Every Bad Value Announces Itself First

Before

Check the shape first.

Today

Try the risky step directly.

Not Every Bad Value Announces Itself First

Before

Check the shape first.

Today

Try the risky step directly.

If it fails

Handle the failure.

Try / Except Is Like Trying the Door Handle

TRY

Attempt the action directly.

Try / Except Is Like Trying the Door Handle

TRY

Attempt the action directly.

EXCEPT

If it fails, react to what happened.

No separate permission check first.

Python Has Many Kinds of Errors

`ValueError`

Today's first example: the value can't be converted as requested.

Different error types

Each name points to a different kind of problem.

Common Python Error Types

`ValueError`

bad value

`TypeError`

wrong type

`NameError`

unknown name

`IndexError`

missing position

`KeyError`

missing key

Name the Exact Exception — Never Use Bare `except:`

Specific

```
except ValueError:
```

Catches only the error you expect.

Bare `except:`

Catches everything — including real bugs you want to see.

CODE ALONG

Open the Code-Along

Open `Module1_Lesson10_error-handling_STUDENT.py`
Run it after every section: `python3` + the file name

Sections, in order:

1. Watch This Crash (watch-only, projected)
2. Catching the Crash
   Same broken code, but this time the code flags it and moves on.
3. Putting It Together: Two `except` Clauses
   Same idea, now applied to every line of a batch file, with two different failure types to catch.

Watch This Crash

python

```python
quantity = "abc"
count = int(quantity)
print("Converted:", count)
```

Catching the Crash

```python
quantity = "abc"

try:
    count = int(quantity)
    print("Converted:", count)
except ValueError:
    print("That's not a valid number.")
```

Putting It Together: Two `except` Clauses

```python
def validate_batch(lines):
    for line in lines:
        try:
            fields = line.split(",")
            print("OK:", fields[1], int(fields[2]))
        except IndexError:
            print("Bad format:", line)
        except ValueError:
            print("Bad quantity:", line)
lines = ["ADD,Chess,4", "REMOVE,Uno", "ADD,Dice,two"]
validate_batch(lines)
```

CODE-ALONG DONE

Back to the Slides: Review

You just typed it and ran it. Now check that it stuck, without the code in front of you.

CHECK FOR UNDERSTANDING

In the door-handle analogy, what's the equivalent of the `except` block?

A Trying the handle

B Walking up to the door

C Reacting once you find out it's locked

ANSWER KEY

Door-Handle Analogy Recall

Correct: Reacting once you find out it’s locked

The `except` block runs only after the attempt fails — just like reacting to a locked door after trying the handle.

QUICK CHALLENGE

With `quantity = "5"`, what does the Catching the Crash code print?

A

`"Converted: 5"`

B

`"That's not a valid number."`

C

Nothing — it still crashes

ANSWER KEY

Valid Input Trace

Correct answer: `"Converted: 5"`

`int("5")` succeeds, so the `except` block never runs. It exists only for when something actually goes wrong.

Watch It Run: `try/except`, Step by Step

Step 1 of 6

quantity holds the text "abc"

```
quantity = "abc"
try:
    count = int(quantity)
    print("Converted:", count)
except ValueError:
    print("That's not a valid number.")
print("Done: on to the next row")
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

QUICK CHALLENGE

Which error does `"5" + 3` raise?

A `ValueError`

B `TypeError`

C `NameError`

ANSWER KEY

Name the Error: `TypeError`

Correct answer: `TypeError`

`+` can’t combine a string and an integer here. The problem is the types, not the value.

QUICK CHALLENGE

A function reads a variable defined only inside another function. Which error?

A

`NameError`

B

`KeyError`

C

`IndexError`

ANSWER KEY

Name the Error: `NameError`

Correct answer: `NameError`

A name that existed only inside another function's local scope isn't visible here.

QUICK CHALLENGE

A Fellow writes `except:` instead of `except ValueError:`. What's the real risk?

A Nothing — it's just shorter

B It only works for `ValueError`

C It silently catches every error

ANSWER KEY

Bare `except:` Risk

Correct: It silently catches every error

A bare `except:` can swallow typos and real bugs you never meant to hide instead of surfacing them for repair.

QUICK CHALLENGE

Which step raises the error for `"REMOVE,Uno"`?

A

```
line.split(",")
```

B

```
fields[2]
```

C

```
int(...)
```

ANSWER KEY

Full Function Trace

Correct: B, `fields[2]`

`split(",")` gives only 2 fields, so reading `fields[2]` raises `IndexError` before `int()` ever runs. That is why `"Bad format"` prints, not `"Bad quantity"`.

Two Ways to Check Before You Trust It

CHECK FIRST

"Look before you leap"

- `.isdigit()` guards the risky step

TRY FIRST

"Try it, don't ask first"

- `try/except` handles failure

TIP

“Check the Quantity First, or Just Try Converting It?”

CHECK FIRST

`.isdigit()` before conversion

TRY FIRST

Convert, then catch failure

Answer with QAR: your first question, your assumption, the risk.

MODEL ANSWER

Model Answer: QAR

QUESTION

Are valid quantities only whole numbers, 0 and up?

ASSUMPTION

Yes, so the bad rows hold words like "ten".

RISK

`int("-3")` converts fine, so try alone lets negatives in.

TURN AND TALK

Why Is One Unhandled Error Worse on 430 Rows?

One malformed line can stop the whole run.

What does that mean for all the rows that come after it?

Key Terms — Recap

Exception

error event

`try / except`

attempt + handle

`ValueError`

bad value

`TypeError`

wrong type

`NameError`

unknown name

`IndexError` / `KeyError`

missing position / key

So Far, We've Covered…

Today

A real `try/except` catching two failure types.

Next

A second function that summarizes the data.